

Morphosyntactic Programming

Case, Mood, and Type-Directed Disambiguation for Turkish-Like Syntax

JOOMY KORKUT, Bloomberg, USA

ALPEREN KELES, University of Maryland, USA

ONUR AKDEMIR, Topsort, USA

We present Kip, a statically typed functional programming language with a syntactic purity discipline, whose surface syntax is in Turkish and whose type checking is guided by Turkish morphology. Kip uses case suffixes to determine which parameter each argument belongs to, so arguments can be given in any order when the cases uniquely identify them. It carries morphological ambiguity through parsing and resolves it during type checking via type-directed constraint solving. Kip also enforces a lightweight effect discipline via syntax: pure definitions are noun phrases, while effectful computations are infinitival verb phrases, and effectful calls appear in the imperative mood.

CCS Concepts: • **Software and its engineering** → **Functional languages**; *Language features*; • **Human-centered computing** → Natural language interfaces.

Additional Key Words and Phrases: linguistics, morphosyntax, grammatical case, grammatical mood, type systems, type-directed disambiguation

ACM Reference Format:

Joomy Korkut, Alperen Keles, and Onur Akdemir. 2026. Morphosyntactic Programming: Case, Mood, and Type-Directed Disambiguation for Turkish-Like Syntax. In *Human Aspects of Types and Reasoning Assistants*. ACM, New York, NY, USA, 10 pages.

1 INTRODUCTION

Most mainstream programming languages use English-oriented keywords and naming conventions, largely due to historical and ecosystem-related reasons rather than technical necessity. As a result, many surface-language design decisions are implicitly English-centric: argument order is primarily positional, meaning is carried by keywords and punctuation rather than by morphological structure. A number of projects have experimented with non-English syntax, but most translate keywords while leaving the underlying structure unchanged—an approach that broader localization research identifies as only one of several relevant design dimensions [24].

Morphologically rich languages such as Turkish, Finnish, Hungarian, German, Russian, and Latin mark grammatical relations directly on words through inflectional suffixes [5]. In Turkish, for instance, case suffixes help indicate whether a noun phrase serves as a subject, direct object, recipient, location, source, modifier, or instrument. English typically encodes these relations through position and prepositions. This raises the question of whether *morphosyntax*—the interplay of word form and syntactic function—can participate directly in static reasoning instead of being discarded after parsing as presentation-only syntax.

We explore this question through Kip,¹ a statically typed functional language whose surface syntax is based on Turkish and whose elaboration is morphology-aware. Rather than normalizing away inflection, Kip preserves case information after parsing and uses it operationally in name resolution, overload selection, and argument-parameter matching. Case annotations are part of types, so the checker can use morphological evidence to match arguments to parameters during type-directed elaboration.

As a brief illustration, a binary function signature in Kip marks each parameter with a case suffix on the type phrase:

¹Kip is Turkish for “grammatical mood.” Kip’s implementation is available open source at <https://github.com/kip-dili/kip>.

Surface	Morphological Gloss
(<i>bu</i> <i>doğal-sayıyla</i>) (<i>şu</i> <i>doğal-sayının</i>) <i>farkı</i> , ...	(<i>this</i> :N:NOM <i>natural</i> :ADJ:NOM-number:N:INS) (<i>that</i> :N:NOM <i>natural</i> :ADJ:NOM-number:N:GEN) <i>difference</i> :N:P3s, ...

Here *doğal-sayıyla* carries the instrumental suffix *-(y)la* (roughly “with a natural number”) and *doğal-sayının* carries the genitive suffix *-(n)in* (“of a natural number”). Because the two cases are distinct, arguments can be matched to parameters by case rather than by position and can be supplied in either order when the match is unique. For a noncommutative function such as difference, this matching matters: the suffixes determine which number fills which parameter. Conversely, when parameters are not distinguishable by case alone—for example, when multiple parameters bear the same case—Kip falls back to ordinary positional interpretation.

The rest of the paper develops this design through three contributions:

- (1) A case-structured interface discipline for functions and constructors, with elaboration rules that match arguments to parameters and permit non-positional application when the match is unique.
- (2) A type-directed account of morphological ambiguity in which analyses from the parser are carried forward and pruned by case and typing constraints, with explicit user-level disambiguation when needed.
- (3) A syntactic purity boundary that maps noun-phrase forms to pure code and infinitive/imperative forms to effectful code, integrated into the static checker and exercised across interactive and file-based programs.

2 SYNTAX THROUGH EXAMPLES

We use three surface forms below: case-marked calls, conditional patterns, and verbal effects. In the examples, **keywords**, **types**, **constructors/literals**, **functions**, and **bound names** are distinguished typographically; glosses use tags such as :GEN (genitive), :INS (instrumental), :ACC (accusative), :COND (conditional), :INF (infinitive), and :IMP (imperative). :P3s marks third-person singular possessive, used here as a nominal-compound linker.

2.1 Case-Marked Interfaces

Pure Kip definitions are noun phrases. A function definition places case-marked arguments before a nominal head:

Surface	Morphological Gloss
(<i>bu</i> <i>tam-sayının</i>) <i>karesi</i> , <i>bunla bunun çarpımıdır</i> .	(<i>this</i> :N:NOM <i>integer</i> :N:GEN) <i>square</i> :N:P3s, <i>this</i> :N:INS <i>this</i> :N:GEN <i>product</i> :N:P3s:COP.

The genitive suffix on *tam-sayının* marks the parameter’s case. At call sites, arguments likewise precede the nominal head:

Surface	Morphological Gloss
(<i>5</i> :in <i>karesini</i>) <i>yaz</i> .	(<i>5</i> :GEN <i>square</i> :N:P3s:ACC) <i>write</i> :v:IMP.

This applies *kare* to 5 and prints the result.

Function headers declare parameters through case-marked type phrases. At a call site, elaboration filters candidate signatures by arity, case compatibility, and type compatibility. If the observed argument cases determine a unique alignment with the signature cases, Kip reorders arguments into signature order before elaborating the call. When multiple parameters bear the same case,

case information alone cannot determine a permutation, so matching falls back to positional order within that repeated-case segment.

Partial application uses the same case information. If fewer arguments than a signature's arity are supplied, Kip fills the uniquely identified parameters and returns a residual function for the remaining parameters:

Surface	Morphological Gloss
<code>(3'le (1'in toplamını)) yaz.</code>	<code>(3:INS (1:GEN sum:N:P3S:ACC)) write:V:IMP.</code>

The inner expression supplies the genitive argument to `toplam`; the outer call supplies the remaining instrumental argument.

2.2 Patterns and Data

Algebraic data types and patterns use the same nominal interface discipline. Pattern branches are marked with Turkish conditional morphology:

Surface	Morphological Gloss
<code>(bu öge listesinin) uzunluğu, bu boşsa, 0; ilkın devama ekiyse, (devamın uzunluğuyla) 1'in toplamıdır.</code>	<code>(this:N:NOM element:N:NOM list:N:P3S:GEN) length:N:P3S, this:N:NOM empty:N:COND, 0:NOM; head:N:GEN rest:N:DAT addition:N:P3S:COND, (rest:N:GEN length:N:P3S:INS) 1:GEN sum:N:P3S:COP.</code>

The conditional suffix `-sa/-se` marks pattern branches. Checking a clause unifies the scrutinee type with the constructor result type and propagates the resulting substitution to bound pattern variables. The implementation also checks exhaustiveness and rejects repeated binder names within a branch.

2.3 Effects

Kip's purity boundary is grammatical rather than annotation-driven: nominal forms denote pure definitions, while infinitival verb forms denote effectful computations. Effectful sequencing uses converbs such as `-ip`, and `için` binds intermediate results:

Surface	Morphological Gloss
<code>selamlamak, isim için okuyup, ("Merhaba "yła ismin birleşimini) yazmaktır. selamla.</code>	<code>greet:V:INF, name:N:NOM for read:V:CVIP, ("Hello ":INS name:N:GEN union:N:P3S:ACC) write:V:INF:COP. greet:V:IMP.</code>

This definition reads a name, writes a greeting, and is invoked imperatively. Imperative calls are effectful commands. There can be multiple of them in the same program; they are not a unique program entry point marker like `main` in Haskell. The same surface distinction is enforced by the checker: pure nominal contexts cannot contain effectful expressions. Thus the morphology that distinguishes noun phrases from verb phrases also marks the static purity boundary.

The examples in this section illustrate the core surface mechanisms: case-marked arguments, conditional morphology for patterns, and verbal mood for effects. The next section examines what happens when morphological information is ambiguous.

3 MORPHOLOGICAL AMBIGUITY AS TYPE-DIRECTED CHOICE

Morphological ambiguity is where the design becomes type-directed. Kip does not commit to a single morphological analysis at parse time; it defers that choice to type checking, where richer context is available.

Turkish morphology produces genuine lexical ambiguity: a single surface form can correspond to distinct root+suffix analyses. The form *kümesi*, for instance, may be read as *küme-si* (“its set”) or as *kümes-i* (“chicken coop” plus case or possessive morphology), yielding a different lexical base and potentially a different case. Kip treats these ambiguities as typed disambiguation problems. The parser queries TRmorph [26, 27] and stores surviving root, case, and part-of-speech analyses in the AST after scope filtering; small analyzer-coverage gaps are handled by predictable case completions. The checker then prunes these candidates by scope, case compatibility, and type constraints, and the programmer can force a reading with apostrophes when context is insufficient. Rejecting all parser ambiguity early would reject programs whose intended reading is uniquely determined by later type and case evidence.

3.1 Type-Directed Resolution

Given a parsed AST whose ambiguous nodes carry candidate analyses, the type checker resolves each node to a unique reading as follows:

- (1) **Name resolution.** Remove analyses whose roots are not in scope; an empty candidate set is an unknown-name error.
- (2) **Call resolution.** At each call site, filter candidate signatures by arity, case compatibility, and type compatibility. For surviving signatures, align observed argument cases with declared parameter cases; if alignment is unique, reorder arguments into signature order, otherwise retain positional order within repeated-case segments.
- (3) **Commitment.** Check effects and exhaustiveness, then require every remaining candidate set to be singleton. Otherwise, report the first ambiguity or type mismatch.

After this pass, every surface form has a unique morphological reading, every call has a unique overload with arguments in signature order, and no effectful expression appears in a pure context.

Kip also supports ordinary type-driven overloading: the same surface head can be defined at multiple types. Overload selection is handled by the same pass as morphological disambiguation: candidate signatures are filtered by arity, case compatibility, and type compatibility, so argument types can select among heads that share a surface form.

3.2 Head-Final Syntax and Parenthesization

Because Turkish is head-final, Kip places the function name after its arguments. This gives pure expressions an RPN-like shape: if every head has a fixed arity, constituent structure is recoverable without parentheses. Overloading weakens that property, since the parser may not know how many preceding arguments a head consumes. Kip therefore permits explicit parentheses, and requires them where overloaded heads create genuine grouping ambiguity. This grouping issue is separate from morphological ambiguity: it concerns constituent attachment, whereas the disambiguation mechanism below concerns which morphological reading and typed signature a constituent receives.

4 DESIGN TRADE-OFFS AND LIMITATIONS

The case-based design rules out several simpler alternatives. We discuss argument assignment, the timing of morphological disambiguation (§4.2), the purity boundary (§4.3), the gap between Kip and ordinary Turkish (§4.4), and the current prototype’s limits.

4.1 Argument Assignment Strategies

The simplest alternative is purely positional argument assignment, where arguments are matched to parameters entirely by order. This yields simple parsing and predictable elaboration, but it weakens one of Kip’s core linguistic affordances: Turkish-like reordering for emphasis and local readability. It also makes morphological case mostly ornamental, since parameter identity is already fixed by position. More subtly, it suppresses focus-sensitive alternatives in Kip interfaces: a membership query can foreground either the set or the element by ordering the dative and genitive arguments differently, while a strictly positional interface would force one canonical wording.

Another option is to keep fixed function names but attach explicit parameter labels (keyword arguments or preposition-like markers) to arguments. This improves clarity and avoids some ambiguity, but introduces a parallel labeling language separate from ordinary case morphology. For Kip, that would reduce the central claim that natural inflection itself can guide argument assignment through static checking.

Kip instead uses grammatical case as the primary cue for argument assignment in function and constructor interfaces. At call sites, arguments may be reordered when cases and types jointly recover a unique argument-to-parameter assignment. This aligns with Turkish morphosyntax and makes case markings computationally relevant rather than merely decorative. The trade-off is increased checker complexity, since argument matching, overload selection, and ambiguity handling become coupled—but studying that coupling is precisely the research goal.

In practice, reordering is most useful when parameters have distinct cases and the distinction is semantically salient (e.g., source vs. destination, accumulator vs. current value). It is less useful when multiple parameters share the same case or when the distinction is already obvious from local context. Accordingly, Kip’s implementation keeps conservative matching heuristics and falls back to more positional behavior in repeated-case situations.

4.2 Early vs. Late Disambiguation

Independent of argument style, language designers must choose when morphological ambiguity is resolved: at lex/parse time (early commitment, simpler later phases) or at elaboration/type-check time (deferred commitment, richer context). Kip chooses deferred commitment.

This choice is not just an implementation detail. Gordon observes that the boundary between syntax and semantics in programming-language implementations is partly conventional: compilers often parse with a context-free grammar and then check scoping and typing in later passes, even though scoping and simple typing can be viewed as context-sensitive language problems [9]. Kip makes a similar boundary choice for morphology. It parses multiple candidate analyses and lets scope, case, and type constraints select among them during elaboration.

This enables type-directed disambiguation but requires diagnostics that explain why one reading survives and another fails. From a localization perspective, this treats morphology as part of the language’s productions and elaboration strategy, rather than as a matter of translated keywords alone [24].

4.3 Purity Boundary Design

Kip’s purity boundary is syntactic: noun-phrase-like forms are pure definitions, while infinitive and imperative forms are effectful. This replaces an explicit effect annotation language in source syntax. The split also avoids an evaluation-order problem created by case-directed reordering. Haskell isolates effects partly because laziness makes effect timing hard to reason about otherwise [19]; Kip faces an analogous issue from argument matching rather than laziness. If effectful arguments could

be freely reordered, programmers would have to know whether effects occur in surface order or in the signature order induced by elaboration.

OCaml labeled arguments illustrate the issue [7]. They decouple call-site order from parameter order, and OCaml does not guarantee that evaluation follows the visible order of labeled arguments:

```
# let f ~x ~y = x - y;;
val f : x:int -> y:int -> int = <fun>
# f ~x:(printf "x"; 5) ~y:(printf "y"; 3);;
yx- : int = 2
# f ~y:(printf "y"; 3) ~x:(printf "x"; 5);;
yx- : int = 2
```

Partial application makes the same issue more visible:

```
# f ~y:(printf "y"; 3) ~x:(printf "x"; 5);;
yx- : int = 2
# (f ~y:(printf "y"; 3)) ~x:(printf "x"; 5);;
xy- : int = 2
```

Kip avoids this by restricting case-directed reordering and partial application to pure expressions, where evaluation order is observationally irrelevant. Effectful computation remains in verbal syntax, where sequencing is explicit.

4.4 Uncanny Valley between Kip and Turkish

Kip’s Turkish surface syntax invites speakers to rely on native grammatical expectations, which creates an “uncanny valley” failure mode: programs can look close enough to ordinary Turkish to invite fluent interpretation, yet be rejected because Kip accepts only a constrained case frame [17]. For example, a Turkish speaker may expect both “subtraction of a and b” and “subtraction of a from b” to be valid paraphrases, but Kip currently treats such alternations as distinct interfaces unless the standard library defines synonyms or explicit parameter mappings. More generally, Kip flattens syntactic, semantic, and lexically selected suffixes into a uniform case-label vocabulary, which works best when a suffix transparently identifies a parameter and less well when ordinary Turkish relies on broader case-frame alternations or lexically idiosyncratic case selection [25].

4.5 Evaluation Strategy and Implementation

Once type checking succeeds, the program is a resolved AST: overloads have been selected, arguments have been reordered into signature order, morphological candidate sets have been reduced to single readings, and purity checks have already run. Both the tree-walking interpreter and the JavaScript backend consume this post-checking AST, so evaluation and code generation do not redo case-directed elaboration.

Kip is implemented in approximately 16,000 lines of Haskell, plus a small C wrapper around TRmorph [26], a finite-state Turkish analyzer built on Foma [11]. The prototype includes a standard library, REPL, LSP server, JavaScript backend, and test suites covering case-structured arithmetic, algebraic data types, pattern matching, higher-order list programs, effectful programs, and larger examples such as a DPLL SAT solver.

4.6 Limits and Non-Goals

Promoting morphology from presentation layer to type-checking signal introduces real costs. Diagnostic design becomes central, because users must understand both type and morphological mismatches simultaneously when a program fails. Unrestricted case flexibility can make overload behavior difficult to predict, requiring conservative matching heuristics in practice. The design transfers most directly to languages with overt argument marking; the usability gains depend on the programmer’s fluency in the target language’s inflectional (or declensional) system.

Kip is not a full Turkish grammar, a free-form natural-language programming interface, a principal global disambiguation algorithm over all parses and overloads, or a full effect system with polymorphic effect inference. Its surface language is deliberately constrained: linguistic material is kept where it improves compositional parsing and static checking, and set aside where it would reduce predictability.

5 RELATED WORK

5.1 Natural Language Programming

Dijkstra’s essay on the “foolishness of natural language programming” is a classic objection to treating ordinary language as programming notation: natural language is ambiguous, context-dependent, and ill-suited to the precision expected of programs [4]. Kip accepts that concern. It is not free-form Turkish, but a formal language that adopts a narrow, typed fragment of Turkish morphosyntax. Gordon offers a complementary perspective: programming-language research can draw more directly on linguistics as the study of form, meaning, and use, rather than treating natural language mainly as text to be processed [9]. Kip follows that direction in a concrete language design: Turkish morphosyntax is not only input text, but part of parsing, elaboration, and static checking.

Natural programming research makes a related user-centered argument. Myers, Pane, and Ko define natural programming systems as languages and environments that work the way users expect, and they advocate studying how people formulate tasks before designing programming constructs [18]. Kip shares the narrower premise that programming notation should respect the way its intended users already structure meaning. Unlike that work, however, Kip does not present a usability study or a new end-user environment; its contribution is technical: linguistically motivated surface distinctions participate in typed elaboration and static disambiguation.

Swidan et al. provide a 12-aspect framework for programming language localization [24], including whether a localized language translates an existing programming language or is designed from scratch, and whether its productions follow the word order of the target natural language. Kip falls on the designed-from-scratch side and pushes the production-level case further by making Turkish morphosyntax part of typed elaboration.

5.2 Non-English Programming Languages

Several projects have explored programming languages with non-English syntax. Among these, Tampio is the closest precedent: it is a Finnish natural-language programming language that uses *libvoikko* for Finnish inflection and compiles to JavaScript [10]. The key difference is that Tampio resolves morphological ambiguity greedily during lexing and parsing, whereas Kip carries multiple analyses into type-directed elaboration. Kip also uses case as part of typed dispatch for function application and enforces a noun/verb purity boundary, rather than using morphology primarily as a parsing and readability aid.

Perligata [3] draws heavily on Latin morphology and vocabulary, but regularizes and invents forms to fit programming needs, making it Latin-inspired rather than philologically faithful. Kip

instead targets consistent, typed program construction rather than a linguistic recasting of an existing language.

5.3 Related Mechanisms in Other Programming Languages

Kip’s case-directed argument-parameter matching is related to labeled and named argument mechanisms in several programming languages, even when those mechanisms are not motivated by natural language.

Many languages allow arguments to be passed out of order via explicit naming or labeling—examples include OCaml’s labeled arguments [7], Ada’s named parameter association [13], Swift’s argument labels [1], Smalltalk’s keyword messages [8], and Common Lisp and Python keyword arguments [21, 23]. In all these cases, argument labels are fixed identifiers—either in the function signature or in the calling convention—rather than inflectional features of the argument expressions themselves.

Kip’s nominal/verbal purity boundary is reminiscent of Meyer’s command-query separation principle in Eiffel [16], which enforces a syntactic distinction between side-effecting procedures and pure queries. In a looser sense it also parallels Haskell’s separation between pure expressions and `IO` actions, with grammatical mood taking on some of the surface work that `do`-notation and monadic types do there. Languages with richer effect systems, such as Koka [14], Eff [2], and Frank [15], instead make effect distinctions explicit through polymorphic effect types and handlers. Kip’s version is deliberately lighter-weight and coarser than these richer effect systems: it uses grammatical mood to enforce a purity boundary without exposing an effect-type language in surface syntax.

The distinction is that Kip reuses a natural language’s existing argument-marking system as part of typed elaboration instead of introducing a separate labeling mechanism.

5.4 Controlled Natural Languages

Kuhn characterizes controlled natural languages (CNLs) as engineered varieties of a base natural language that restrict lexicon, syntax, and/or semantics to improve comprehensibility, translation quality, or formal processability [12]. Kip overlaps with this tradition, especially CNLs that map restricted natural-language input to executable or formally interpretable representations [6, 20, 22]. The methodological direction is different: CNLs typically start from ordinary-language-like sentences and constrain them until interpretation is precise, whereas Kip starts from typed elaboration requirements and admits linguistic devices only where they improve static reasoning. Kip is therefore complementary to CNL work rather than a canonical CNL.

6 CONCLUSION

Kip shows one way for natural-language morphology to enter static semantics instead of merely decorating a programming-language surface. Case marking supports argument-parameter matching, word order can be relaxed when the matching remains unique, ambiguous analyses are settled during typing, and the noun/verb distinction provides a lightweight syntactic purity boundary.

For localization, this points away from keyword translation and toward elaboration design. Future work will evaluate whether case-marked interfaces improve readability for Turkish-speaking programmers and develop richer ambiguity diagnostics that explain why alternative readings fail.

ACKNOWLEDGMENTS

AI tools were used to reword sentences idiomatically, to check the grammar and spelling, and in the coding of the Kip implementation.

REFERENCES

- [1] Apple Inc. 2026. Functions. <https://docs.swift.org/swift-book/documentation/the-swift-programming-language/functions/>. In *The Swift Programming Language*. Accessed: 2026-02-25.
- [2] Andrej Bauer and Matija Pretnar. 2015. Programming with Algebraic Effects and Handlers. *Journal of Logical and Algebraic Methods in Programming* 84, 1 (2015), 108–123.
- [3] Damian Conway. [n.d.]. Lingua::Romana::Perligata – Perl for the XXI-imum Century. <https://web.archive.org/web/20241217212902/https://users.monash.edu/~damian/papers/HTML/Perligata.html>. Accessed: 2025-12-16.
- [4] Edsger W Dijkstra. 2005. On the foolishness of “natural language programming”. In *Program Construction: International Summer School*. Springer, 51–53.
- [5] Matthew S. Dryer and Martin Haspelmath (Eds.). 2013. *WALS Online (v2020.4)*. Zenodo. <https://doi.org/10.5281/zenodo.13950591>
- [6] Norbert E. Fuchs and Rolf Schwitter. 1995. Specifying Logic Programs in Controlled Natural Language. In *Proceedings of the Workshop on Computational Logic for Natural Language Processing (CLNLP 1995)*. Edinburgh, Scotland, UK. April 3–5, 1995.
- [7] Jacques Garrigue. 2001. Labeled and optional arguments for Objective Caml. In *JSSST Workshop on Programming and Programming Languages, Kameoka, Japan*.
- [8] Adele Goldberg and David Robson. 1983. *Smalltalk-80: The Language and its Implementation*. Addison-Wesley.
- [9] Colin S. Gordon. 2024. The Linguistics of Programming. In *Proceedings of the 2024 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software* (Pasadena, CA, USA) (Onward! '24). Association for Computing Machinery, New York, NY, USA, 162–182. <https://doi.org/10.1145/3689492.3689806>
- [10] Iikka Hauhio. [n.d.]. Tampio Programming Language (GitHub Repository). <https://github.com/fergusq/tampio>. Repository and README. Accessed: 2026-02-26.
- [11] Mans Hulden. 2009. Foma: a finite-state compiler and library. In *Proceedings of the 12th Conference of the European Chapter of the Association for Computational Linguistics*. Association for Computational Linguistics, 29–32.
- [12] Tobias Kuhn. 2014. A survey and classification of controlled natural languages. *Computational linguistics* 40, 1 (2014), 121–170.
- [13] Henry Ledgard. 1981. *ADA: An Introduction, Ada Reference Manual (July 1980)*. Springer-Verlag.
- [14] Daan Leijen. 2014. Koka: Programming with Row Polymorphic Effect Types. *Electronic Proceedings in Theoretical Computer Science* 153 (2014), 100–126.
- [15] Sam Lindley, Conor McBride, and Craig McLaughlin. 2017. Do Be Do Be Do. In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages (POPL)*. ACM, 500–514.
- [16] Bertrand Meyer. 1988. *Object-Oriented Software Construction*. Prentice Hall.
- [17] Masahiro Mori, Karl F. MacDorman, and Norri Kageki. 2012. The Uncanny Valley. *IEEE Robotics & Automation Magazine* 19, 2 (June 2012), 98–100. <https://doi.org/10.1109/MRA.2012.2192811> English translation of Mori's (1970) essay.
- [18] Brad A. Myers, John F. Pane, and Amy J. Ko. 2004. Natural programming languages and environments. *Commun. ACM* 47, 9 (Sept. 2004), 47–52. <https://doi.org/10.1145/1015864.1015888>
- [19] Simon Peyton Jones. 2001. Tackling the Awkward Squad: Monadic Input/Output, Concurrency, Exceptions, and Foreign-language Calls in Haskell. *NATO Science Series Sub Series III Computer and Systems Sciences* 180 (2001), 47–96. <https://web.archive.org/web/20221212192523/https://simon.peytonjones.org/assets/pdfs/tackling-awkward-squad.pdf>
- [20] Ian Pratt-Hartmann. 2003. A two-variable fragment of English. *Journal of Logic, Language and Information* 12, 1 (2003), 13–45.
- [21] Python Software Foundation. 1997. *Python Reference Manual* (1.5 ed.). Section 5.3.4 (Calls). Accessed: 2026-02-26.
- [22] Rolf Schwitter. 2002. English as a formal specification language. In *Proceedings. 13th International Workshop on Database and Expert Systems Applications*. IEEE, 228–232.
- [23] Guy L. Steele. 1984. *Common LISP: The Language*. Digital Press.
- [24] Alaaeddin Swidan and Felienne Hermans. 2023. A Framework for the Localization of Programming Languages. In *Proceedings of the 2023 ACM SIGPLAN International Symposium on SPLASH-E (Cascais, Portugal) (SPLASH-E 2023)*. Association for Computing Machinery, New York, NY, USA, 13–25. <https://doi.org/10.1145/3622780.3623645>
- [25] Ellen Woolford. 2006. Lexical case, inherent case, and argument structure. *Linguistic inquiry* 37, 1 (2006), 111–130. <https://doi.org/10.1162/002438906775321175>
- [26] Çağrı Çöltekin. 2010. A Freely Available Morphological Analyzer for Turkish. In *Proceedings of the 7th International Conference on Language Resources and Evaluation (LREC 2010)*. 820–827. <http://www.lrec-conf.org/proceedings/lrec2010/summaries/109.html>
- [27] Çağrı Çöltekin. 2014. A Set of Open Source Tools for Turkish Natural Language Processing. In *Proceedings of the Ninth International Conference on Language Resources and Evaluation (LREC'14)*, Nicoletta Calzolari, Khalid Choukri, Thierry

Declerck, Hrafn Loftsson, Bente Maegaard, Joseph Mariani, Asuncion Moreno, Jan Odijk, and Stelios Piperidis (Eds.). 1079–1086. http://www.lrec-conf.org/proceedings/lrec2014/pdf/437_Paper.pdf